

**Stainless Steel 316 Stress Corrosion Cracking System
Technical Design, Operations, and Maintenance Manual**

Document Number: SCC-TDM-001

Revision: 0.1 Draft

Author: John Jin

Date: June 9, 2026

Revision History:

Revision	Date	Author	Description
0.1	June 9, 2026	John Jin	Initial Draft

Table of Contents

1. Introduction	5
1.1. Purpose	5
1.2. Scope.....	6
1.3. Objectives.....	6
2. System Overview.....	7
2.1. Functional Requirements.....	7
2.2. Performance Requirements	7
3. System Overview.....	8
3.1. Architecture	8
3.2. Hardware Components.....	9
3.3. Software Components	9
4. Hardware Design.....	10
4.1. Hardware Block Diagram	10
4.2. Sensor System	12
4.3. Shield Circuit	12
5. Software Design	17
5.1. Firmware Architecture	17
5.2. Operating States.....	17
5.3. Event Codes	19
5.4. MQTT Topics	19
6. Database Design	20
6.1. Database Overview	20
6.2. Process Data Table	21
6.3. Supporting Tables.....	22
6.4. Relationships	23
6.5. Data Indexing Strategy	23
6.6. Data Retention Policy	24

7. Control System Design	25
7.1. Control Objectives.....	25
7.2. Current Control Architecture	25
7.3. Control Modes.....	26
7.4. Safety Logic.....	26
7.5. Pump Aware Compensation.....	27
7.6. Advisory Model Predictive Control (MPC)	27
7.7. Machine Learning Roadmap	28
8. Safety Analysis.....	30
8.1. Hazards.....	30
8.2. Safety Features	31
8.3. Emergency Shutdown Procedure.....	32
8.4. Safety verification Tests.....	32
8.5. Residual Risk and Recommendation	33
9. Verification and Validation.....	35
9.1. Verification and Validation Overview	35
9.2. Unit Tests	35
9.3. Integration and Hardware Validation Procedures	37
9.4. Current Verification Coverage Gaps	38
9.5. Verification Summary	39
10. Deployment Guide	40
10.1. Deployment Overview	40
10.2. Server Requirements	40
10.3. Containerized Services	41
10.4. Deployment Procedure	41
10.5. Post-Deployment Verification.....	43
11. Operations Guide.....	44
11.1. Startup Procedure.....	44
11.2. Corrosive Medium Preparation and Handling	44
11.3. Operator Responsibilities and Monitoring Requirements	45

Operator Responsibilities	45
Monitoring Requirements.....	45
11.4. Alarm Response Procedures	46
11.5. Shutdown Procedure.....	47
12. Maintenance Manual.....	48
12.1. Maintenance Overview	48
12.2. Preventive Maintenance	48
12.3. Corrective Maintenance	49
12.4. Component Replacement Guidelines	50
12.5. Maintenance Safety Considerations	50
13. Troubleshooting Guide	52
13.1. Troubleshooting Overview	52
13.2. Troubleshooting Workflow	55
13.3. Escalation Criteria	56
14. References	57

1. Introduction

1.1. Purpose

The Stress Corrosion Cracking (SCC) test system is an automated control system designed to heat the specimen and pump corrosive medium, such as magnesium chloride (MgCl_2), while the sample undergoes creep loading. The platform integrates embedded hardware, process control algorithms, wireless communication, database infrastructure, and data analysis into a unified architecture [1–10].



Fig 1. Testing setup

1.2. Scope

This document will include the following facets of the system:

- System architecture
- Hardware design
- Firmware design
- Backend infrastructure
- Database structure
- Control algorithms
- Safety systems
- Testing procedures
- Operational procedures

1.3. Objectives

The primary objectives of the SCC platform are:

1. **Reliable process**

The system must be designed to repeat the experiment process repeatedly. This process should be capable of being repeated with minimal human intervention to address any potential issues that may arise.

2. **Reliable monitoring method**

The system must be monitored with ease and reliability. All sensors must remain operational for an extended period beyond the duration of a single testing experiment.

3. **Automated test control (temperature, pump frequency, etc.)**

The PID system should be equipped with self-tuning capabilities and housed in a secure environment.

4. **Safe heater operation**

The temperature should not exceed 20 degrees Celsius above the set point or 350 degrees Celsius.

5. **Continuous data logging**

It's important to log crucial parameters, including temperature, pump cycle, and time, in CSV format. These parameters encompass any data required for stress corrosion testing analysis, physics-informed machine learning models, or digital twins.

6. **Remote monitoring capability**

The parameters of the heater and pump should be adjustable remotely via the cloud. Firmware for both ESP32 and Arduino Uno R3 should support Over the Air (OTA) updates. Additionally, the Kill Command should also be executed via OTA.

2. System Requirements

This system is employed for conducting stress corrosion cracking experiments. It provides heating and a corrosive medium for the dog-bone samples during creep loading. The cartridge heater enables users to set the desired temperature, while the peristaltic pump propels MgCl_2 (corrosive medium) through the copper pipes and cells, exposing the samples to the corrosive environment. The entire system should function in a quasi-automated mode.

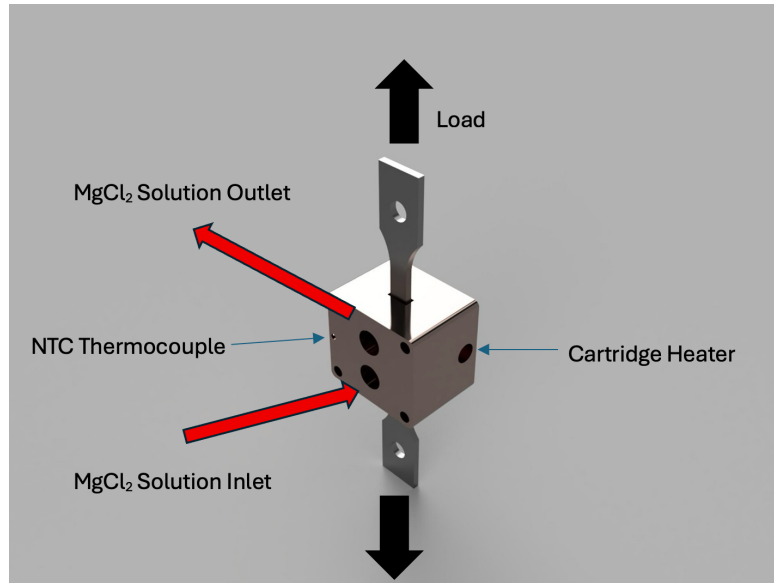


Fig 2. The hardware schematic of the system

2.1. Functional Requirements

ID	Requirement
FR-001	Measure system temperature
FR-002	Control heater output
FR-003	Control pump circulations
FR-004	Log data
FR-005	Provide a method to monitor remotely
FR-006	Execute safety shutdowns

Table 1. List of required functions for the system to perform

2.2. Performance Requirements

ID	Requirement	Target
PR-001	Telemetry update rate	1Hz
PR-002	Data retention	≥ 1 year
PR-003	Uptime	>99%
PR-004	Network Latency	<2 s

Table 2. List of performance targets the system needs to achieve

3. System Overview

3.1. Architecture

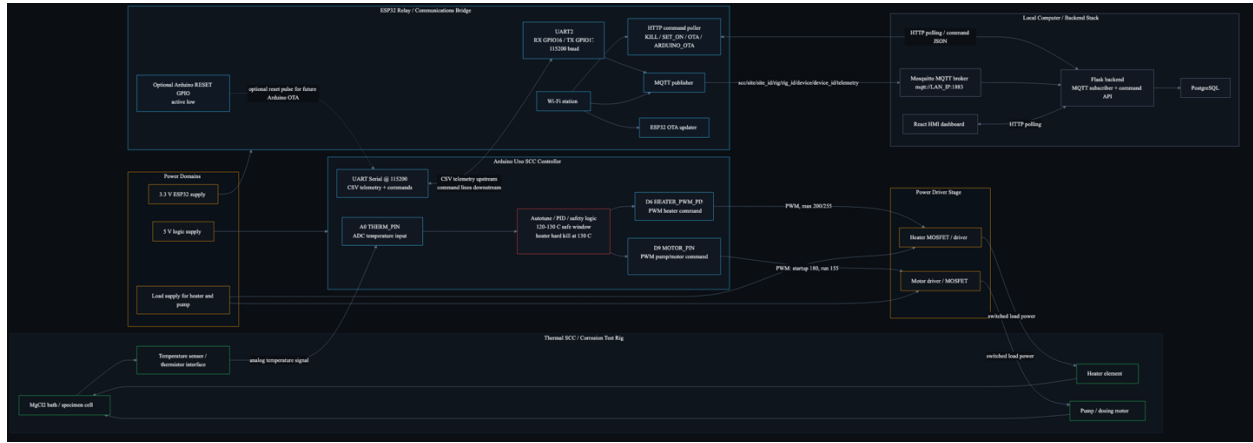


Fig 3. End-to-end system architecture

Fig 3 illustrates the end-to-end architecture of the SCC testing platform. The NTC thermistor and cartridge heater interface with the Arduino controller, which performs local control and data acquisition functions using the hardware components summarized in Table 3. The Arduino communicates with an ESP32 module that serves as a telemetry and command relay between the field hardware and backend infrastructure.

Sensor measurements, process data, and control messages are transmitted through the MQTT broker to the Flask API, which acts as the central integration layer for data management, command processing, analytics, and operator interaction. The software components that support these functions are summarized in Table 4. Processed data are stored in a PostgreSQL database, enabling long-term data retention, historical analysis, and performance monitoring.

The backend provides additional services, including alarm generation, pump-cycle extraction, machine-learning-based prediction, and advisory Model Predictive Control (MPC). Real-time communication between the backend and operator interface is implemented using a Socket.IO event bus [10]. This continuously updates the Human–Machine Interface (HMI) dashboard with process status, alarms, trends, and operator recommendations.

Operator-issued commands are routed through the Flask API and command store to the ESP32 command bridge, enabling remote actuation and over-the-air (OTA) firmware deployment. By separating field control, data acquisition, analytics, data storage, and user interaction into modular subsystems, the proposed architecture improves system scalability, maintainability, reliability, and future extensibility.

3.2. Hardware Components

The SCC testing platform consists of several hardware subsystems responsible for sensing, actuation, control, and communication. The primary hardware components and their functions are summarized in Table 3.

Component	Function
Arduino	Embedded controller
ESP32	Telemetry and command relay
Thermistor	Temperature measurement
Cartridge heater	Thermal control
Pump	Fluid Circulation
SCC Shield	Power switching

Table 3. List of hardware in the system

3.3. Software Components

The software architecture is organized into modular services responsible for control, communication, data management, and user interaction using MQTT, Flask, PostgreSQL, Socket.IO, and Docker technologies [1,4–10]. The major software components used in the SCC platform are summarized in Table 4.

Components	Function
Arduino Firmware	Local Control
ESP32 Firmware	Relay telemetry and commands
MQTT Broker	Messaging
Flask API	Backend services
Flask Server	Data Login and HMI interface
PostgreSQL	Data Storage
Dashboard	User interface HMI

Table 4. List of software tools used in the system

4. Hardware Design

4.1. Hardware Block Diagram

Fig 4 shows the hardware block diagram of the SCC testing platform. The system is centered around an Arduino UNO R3 microcontroller, which executes the local temperature control and pump management logic. The Arduino interfaces with a custom SCC Shield that provides sensor acquisition, power switching, and hardware interfacing functions. An NTC thermistor is connected to the SCC Shield for temperature measurement, while a cartridge heater and peristaltic pump are controlled through the shield to regulate the thermal and fluid circulation conditions of the SCC test environment. An ESP32 module serves as a communication bridge between the embedded controller and the supervisory control system. Through a Wi-Fi connection, the ESP32 exchanges telemetry and control commands with a remote Flask server, enabling real-time monitoring, data logging, and remote operation of the testing platform. This architecture separates local control functions from supervisory communication tasks, improving system modularity, maintainability, and scalability.

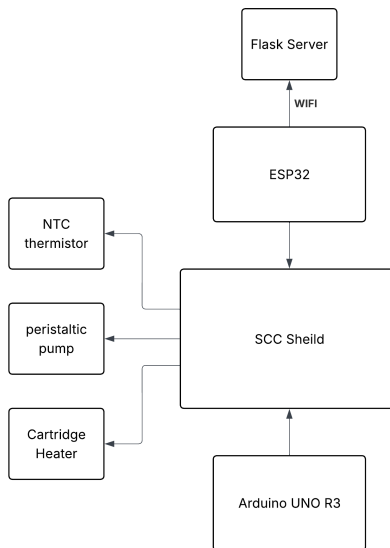


Fig 4. Hardware Block Diagram of the SCC Testing Platform

- **Arduino Uno R3**

Purpose: The Arduino Uno R3 serves as the primary embedded controller for the SCC testing platform and is based on the ATmega328P microcontroller architecture [3,15]. It acquires temperature measurements from the NTC thermistor, executes local control logic, manages heater and pump operation, and communicates process data to the ESP32 communication module.

Key Spec:

- Microcontroller: ATmega328P

- Operating Voltage: 5 V
- Clock Speed: 16 MHz
- Digital I/O Pins: 14
- Analog Input Channels: 6
- Communication Interfaces: UART, I²C, SPI

Firmware version: SCC V1.4

- **ESP32**

Purpose: The ESP32 functions as the communication bridge between the embedded control hardware and the supervisory control system using integrated Wi-Fi communication capabilities [2]. It receives telemetry data from the Arduino, publishes process information through MQTT, receives remote commands, and supports over-the-air (OTA) firmware updates.

Key Spec:

- Processor: Dual-core Xtensa LX6
- Clock Speed: Up to 240 MHz
- Wireless Connectivity: 2.4 GHz Wi-Fi and Bluetooth
- Flash Memory: 4 MB
- Communication Interfaces: UART, SPI, I²C

Firmware version: ESP32-Relay

- **Intel NUC**

Purpose: The Intel NUC serves as the local supervisory computer hosting the Flask backend, MQTT broker, PostgreSQL database, and analytics services [14], MQTT broker, PostgreSQL database, data historian, alarm processing services, machine learning models, and Human–Machine Interface (HMI).

Key Spec:

- Processor: Intel Core Series Processor
- Operating System: Ubuntu Linux 24.04 LTS
- Memory: 8 GB DDR4 RAM
- Storage: 125 GB SSD
- Network Connectivity: Gigabit Ethernet and Wi-Fi

Firmware version: SCC Control V0.1.1.4

4.2. Sensor System

- **Temperature Sensor:**

Type: Negative Temperature Coefficient (NTC) Thermistor

Purpose:

The NTC thermistor provides continuous temperature measurement of the SCC test environment. The Arduino controller uses the sensor to regulate heater operation, monitor process conditions, and provide temperature feedback for data logging and future advanced control applications.

Sampling Method:

Analog voltage measurement through the Arduino Uno R3 analog-to-digital converter (ADC).

4.3. Shield Circuit

Overview of system shield design:

The SCC Shield is a custom-designed interface and power-control board that serves as the hardware integration layer between the Arduino Uno R3 and the SCC test rig components. The shield provides sensor signal conditioning, power distribution, and actuator control functions required for automated operation of the corrosion testing platform. Temperature measurements from the NTC thermistor are routed through an analog conditioning circuit and supplied to the Arduino's analog-to-digital converter (ADC) for real-time temperature monitoring. High-current switching circuitry on the shield enables the Arduino to safely control the cartridge heater and peristaltic pump using low-voltage digital control signals. The shield also provides electrical isolation between low-power control electronics and higher-power load circuits, improving system reliability and protecting the microcontroller from electrical transients generated by inductive loads.

In addition to power-switching functionality, the SCC Shield consolidates wiring connections into a single interface board, simplifying system assembly, maintenance, and troubleshooting. The modular design allows sensors, actuators, and communication hardware to be connected through standardized connectors while maintaining compatibility with the Arduino platform. This architecture reduces wiring complexity and provides a scalable foundation for future expansion of sensing, control, and data acquisition capabilities.

Schematic:



Board Layout:

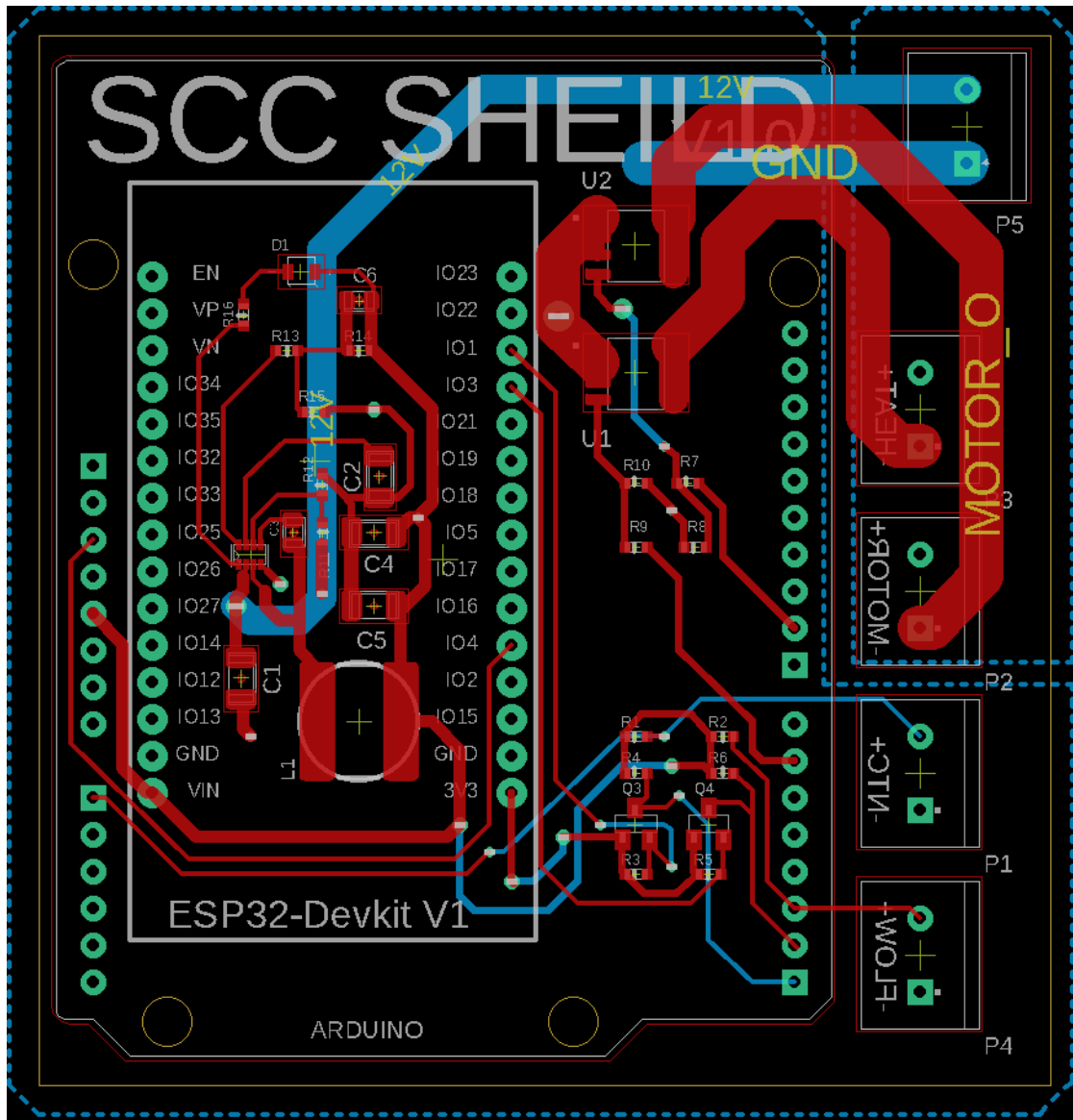


Fig 6. SCC shield board layout

- **Heater Control Circuit**

Description:

The heater control circuit provides low-side switching of the cartridge heater using an N-channel MOSFET (U1, FDS6630A). A digital output from the Arduino drives the MOSFET gate through a 10 k Ω gate resistor (R9), allowing the microcontroller to enable or disable heater power. A 1 M Ω pull-down resistor (R10) ensures that the MOSFET remains off during startup, reset conditions, or when the control signal is disconnected. The heater is supplied from the 12 V power rail and is switched under software control to maintain the desired SCC test temperature. This configuration provides efficient

switching with minimal power dissipation while electrically isolating the microcontroller from the heater load current.

Operating Voltage: 12 VDC

Maximum Current: Determined by MOSFET rating and power connector limitations

Switching Method: Digital low-side MOSFET switching

Protection Features:

- Gate pull-down resistor to prevent unintended activation
- Logic-level MOSFET operation
- Separation of control and power paths
- *Pump Control Circuit*

- **Pump Control Circuit**

Description:

The pump control circuit uses a second N-channel MOSFET (U2, FDS6630A) to control power delivery to the peristaltic circulation pump. The Arduino drives the MOSFET gate through resistor R7, while pull-down resistor R8 maintains the MOSFET in an off state when no control signal is present. The pump receives power from the 12 V supply and is activated periodically according to the SCC control algorithm to circulate solution through the test apparatus. The MOSFET-based design enables reliable switching while minimizing mechanical wear associated with relay-based control methods.

CAUTION: Use a minimum **14 AWG wire** for all motor power connections. The peristaltic pump can draw significant startup and operating current, and undersized wiring may result in excessive voltage drop, overheating, reduced motor performance, or potential damage to wiring and connectors.

Operating Voltage: 12 VDC

Current Draw: Dependent on pump operating conditions

Switching Method: Digital low-side MOSFET switching

Protection Features:

- Gate pull-down resistor to prevent unintended activation
- Logic-level MOSFET operation
- Dedicated power routing for actuator loads

5. Software Design

5.1. Firmware Architecture

Fig 7 presents the operating state machine implemented within the SCC testing platform firmware. The control logic is organized into six primary operating states: Startup, Idle, Heating, Pumping, Lockout, and Emergency Shutdown. Measured temperature, user-defined setpoints, pump scheduling requirements, and safety conditions determine state transitions. During normal operation, the controller transitions from Startup to Idle, where system readiness is verified. When the measured temperature falls below the configured setpoint, the controller enters the Heating state and activates the cartridge heater. Once the target temperature is reached, the system returns to Idle and periodically executes Pumping cycles to circulate the test solution. Safety monitoring is continuously active throughout all operating states. If an abnormal temperature drop is detected following a pump cycle, the controller enters Lockout mode to prevent ineffective heating and excessive actuator cycling. Critical faults, including excessive temperature, sensor failure, or manual emergency stop commands, force the system into Emergency Shutdown mode, where all outputs are disabled until operator intervention occurs. This state-based architecture improves system reliability, predictability, and operational safety by clearly separating normal control functions from fault-handling behavior.

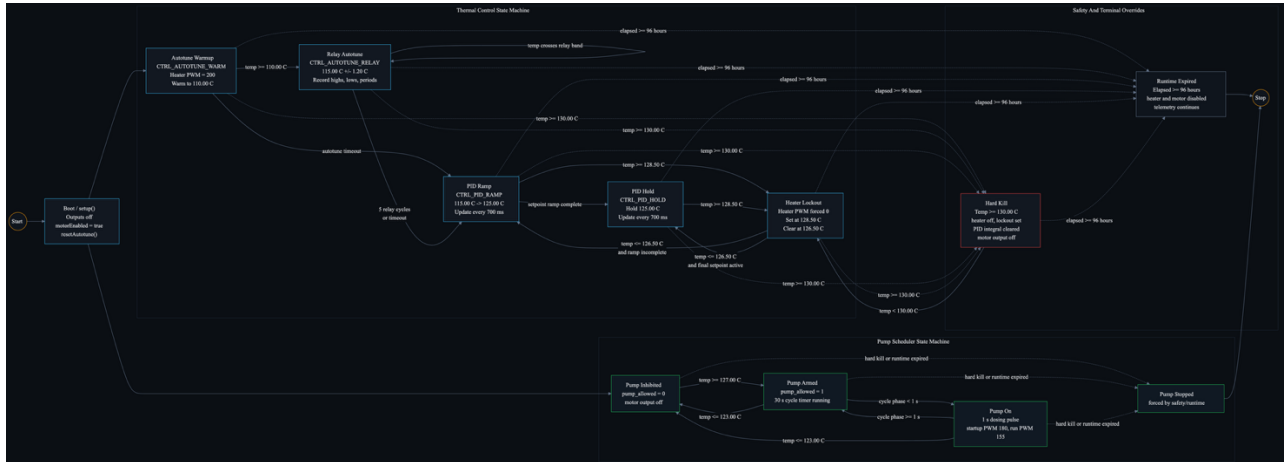


Fig7. SCC Control State Machine Annotation

5.2. Operating States

1. Startup

The Startup state is entered immediately after power-up or system reset. During this phase, the controller initializes hardware peripherals, verifies sensor communication, loads configuration parameters, and establishes communication with the ESP32

telemetry module. All outputs remain disabled until initialization is successfully completed. Upon successful startup, the controller transitions to the Idle state.

2. Idle

The Idle state represents the normal standby operating condition. In this state, the controller continuously monitors process temperature, evaluates pump scheduling timers, and checks safety conditions while maintaining all actuators in a safe state. If the measured temperature falls below the desired setpoint, the system transitions to the Heating state. If a scheduled circulation cycle is required and operating conditions permit, the controller transitions to the Pumping state.

3. Heating

The Heating state regulates the cartridge heater to raise the process temperature toward the user-defined setpoint. Temperature measurements from the NTC thermistor are continuously monitored to prevent overheating and ensure stable thermal control. When the target temperature is achieved, the heater is disabled and the controller returns to the Idle state. Safety monitoring remains active throughout the heating process.

4. Pumping

The Pumping state activates the circulation pump for a predefined duration to ensure fluid mixing and representative temperature measurements. During pump operation, the controller records thermal response characteristics, including temperature drop and recovery behavior. Upon completion of the pump cycle, the system returns to the Idle state. If abnormal cooling or unsafe operating conditions are detected, the controller may transition to the Lockout or Emergency Shutdown state.

5. Lockout

The Lockout state is a protective operating mode used to prevent repeated heating and pumping cycles under unfavorable conditions. Lockout may be triggered when the temperature fails to recover adequately after a pump cycle or when operating limits defined by the control strategy are exceeded. While in Lockout, heater and pump operations are restricted until acceptable process conditions are restored. Once recovery criteria are met, the controller automatically returns to the Idle state.

6. Emergency Shutdown

The Emergency Shutdown state is the highest-priority safety state. It is entered whenever a critical fault condition is detected, such as excessive temperature, sensor failure, communication failure, hardware fault, or a manual emergency stop command. Upon entry, all actuators are immediately disabled and an emergency event is logged and transmitted to the backend system. Recovery from Emergency Shutdown requires operator acknowledgement and corrective action before normal operation can resume.

5.3. Event Codes

The SCC firmware records key operational events using a standardized event-code system to support data logging, diagnostics, and backend analytics. Each event is transmitted through the telemetry interface and stored within the database, allowing system behavior to be reconstructed during testing and fault investigation. The event codes used by the platform are summarized in table 5.

Code	Event
0	Normal Sample
1	Pump start
2	Pump end
3	Recovery complete
4	Hard kill

Table 5. Event codes generated by the SCC firmware for operational logging and system monitoring.

5.4. MQTT Topics

MQTT serves as the primary messaging layer between the ESP32 gateway and the backend infrastructure and enables lightweight publish-subscribe communication for industrial telemetry applications [1,7]. Topic namespaces are structured using site, rig, and device identifiers to support multi-device deployments and simplify message routing. Telemetry topics provide real-time process data, while dedicated status, alarm, heartbeat, and OTA topics support operational monitoring, fault management, and firmware lifecycle control. The MQTT topic definitions used by the SCC platform are listed in Table 6.

Topic	Direction	Description
scc/site/<site_id>/rig/<rig_id>/device/<device_id>/telemetry	Publish	Live ESP32 telemetry
scc/site/<site_id>/rig/<rig_id>/device/<device_id>/	Publish	Device state
scc/site/<site_id>/rig/<rig_id>/device/<device_id>/alarm	Publish	Device alarms
scc/site/<site_id>/rig/<rig_id>/device/<device_id>/heartbeat	Publish	Device heartbeat
scc/site/<site_id>/rig/<rig_id>/device/<device_id>/ota/status	Publish	OTA status

Table 6. MQTT topics used for device telemetry, status reporting, alarm notification, heartbeat monitoring, and OTA firmware management within the SCC platform.

6. Database Design

6.1. Database Overview

The SCC platform utilizes a PostgreSQL database as the central repository for process telemetry, alarms, events, and predictive analytics data [4]. MQTT message logs, system events, alarms, pump-cycle statistics, machine-learning (ML) prediction outputs, model predictive control (MPC) recommendations, and firmware update records. Telemetry data are generated by the Arduino controller, transmitted through the ESP32 communication gateway using MQTT, processed by the Flask backend, and stored within the database for visualization, analytics, alarm generation, and decision-support functions.

The database architecture provides persistent storage for both real-time and historical process information, enabling trend analysis, fault investigation, predictive modeling, and long-term system performance evaluation.

Database Engine: PostgreSQL

Backend ORM: SQLAlchemy [9]

Primary Data Source: ESP32 MQTT Telemetry

Primary Data Table: telemetry

Fig 8 illustrates the database architecture and data flow implemented within the SCC platform.

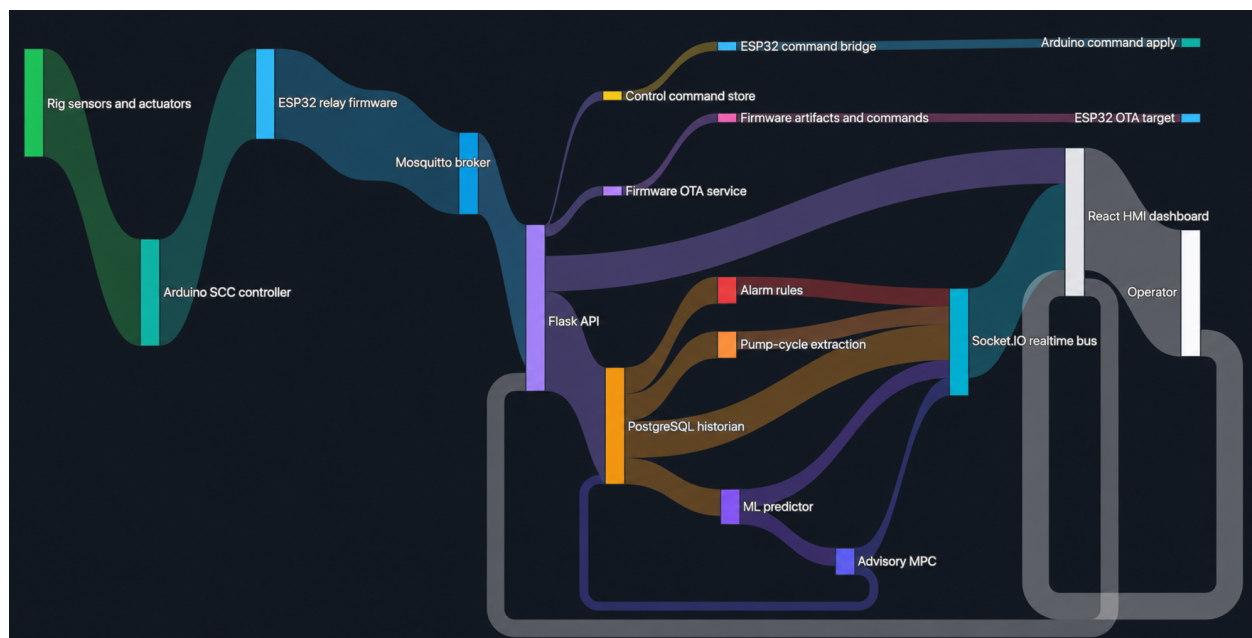


Fig 8. SCC database flow chart

6.2. Process Data Table

The telemetry table serves as the primary repository for process measurements and controller status information. Each telemetry record represents a complete snapshot of system operation at a specific point in time, including temperature measurements, controller states, pump metrics, safety indicators, and operational events. This structure enables historical reconstruction of system behavior, alarm analysis, machine-learning model training, and development of advisory control strategies.

The telemetry table schema is summarized in Table 7.

Table 7. Telemetry table schema used for storage of process measurements, controller states, safety indicators, and pump-cycle metrics within the SCC platform.

Field	Type	Key	Description
id	SERIAL	Primary Key	Unique telemetry record identifier
created_at	TIMESTAMPTZ	Indexed	Timestamp when the telemetry sample was stored
site_id	VARCHAR(80)	Indexed	Site identifier
rig_id	VARCHAR(120)	Indexed	Test rig identifier
device_id	VARCHAR(120)	Indexed	ESP32 device identifier
mqtt_topic	VARCHAR(255)		MQTT topic associated with the received message
raw_payload	JSONB		Original MQTT payload stored for traceability and debugging
event	INTEGER		Event code generated by the controller
ms	INTEGER		Controller runtime in milliseconds
temp_c	DOUBLE PRECISION		Measured process temperature (°C)
adc	INTEGER		Raw analog-to-digital converter value
dtemp_c_per_s	DOUBLE PRECISION		Temperature rate of change (°C/s)
setpoint_c	DOUBLE PRECISION		Temperature setpoint (°C)
mode	INTEGER		Controller operating mode

heater_pwm	INTEGER		Heater PWM duty cycle command
heating	INTEGER		Heater active status flag
heater_lockout	INTEGER		Heater lockout status flag
pump_enabled	INTEGER		Pump enabled status flag
pump_allowed	INTEGER		Pump permission status flag
pump_on	INTEGER		Pump running status flag
motor_pwm	INTEGER		Pump motor PWM duty cycle
motor_on_ms	INTEGER		Pump activation duration (ms)
motor_period_ms	INTEGER		Pump cycle period (ms)
temp_before_pump_c	DOUBLE PRECISION		Temperature immediately before pump activation (°C)
min_temp_after_pump_c	DOUBLE PRECISION		Minimum temperature observed after pump activation (°C)
last_pump_drop_c	DOUBLE PRECISION		Temperature drop caused by the most recent pump cycle (°C)
recovery_time_s	DOUBLE PRECISION		Temperature recovery time following a pump cycle (s)
manual_kill	INTEGER		Manual emergency stop status flag
hard_kill	INTEGER		Hard safety shutdown status flag
uptime_s	INTEGER		Device uptime in seconds

Table 7. Telemetry table schema used for storage of process measurements, controller states, safety indicators, and pump-cycle metrics within the SCC platform.

The raw_payload field is stored as a PostgreSQL JSONB object to preserve the original MQTT message structure and provide traceability for debugging, validation, and future schema extensions.

6.3. Supporting Tables

In addition to the primary telemetry table, several supporting tables are used to manage events, alarms, firmware updates, machine-learning outputs, and control commands. These tables provide separation of concerns while maintaining relationships with the primary telemetry records.

Table 8. Supporting database tables and their functions.

Table	Purpose
events	Stores pump start, pump end, recovery, and hard-kill events
alarms	Stores active and cleared alarm records

pump_cycles	Stores pump-cycle recovery and temperature-drop statistics
devices	Tracks device identity, online status, heartbeat, and MQTT activity
mqtt_message_log	Stores received MQTT messages and validation status
ml_predictions	Stores machine-learning temperature predictions
mpc_recommendations	Stores advisory heater PWM recommendations
control_commands	Stores backend setpoint commands
firmware_devices	Tracks firmware status for connected devices
firmware_update_events	Logs firmware and OTA update events
firmware_artifacts	Stores uploaded firmware artifact metadata
firmware_commands	Stores firmware update commands and acknowledgements

Table 8. Supporting database tables and their functions.

6.4. Relationships

Relationships between tables are implemented using foreign-key references to maintain data integrity and support efficient querying of historical records. These relationships allow telemetry records to be linked with events, alarms, predictive models, firmware operations, and control actions.

The primary database relationships are summarized in Table 9.

Table 9. Database relationships used within the SCC platform.

Relationship	Description
events.telemetry_id -> telemetry.id	Links an event to the telemetry sample that produced it
alarms.telemetry_id -> telemetry.id	Links an alarm to the telemetry sample that triggered it
ml_predictions.telemetry_id -> telemetry.id	Links ML predictions to the source telemetry sample
firmware_update_events.device_id -> firmware_devices.device_id	Links firmware events to a firmware device
firmware_commands.artifact_id -> firmware_artifacts.id	Links firmware commands to uploaded firmware artifacts

Table 9. Database relationships used within the SCC platform.

6.5. Data Indexing Strategy

Database indexes are applied to frequently queried fields to improve query performance for dashboard visualization, historical analysis, alarm investigation, and device monitoring. Indexed columns are selected based on expected filtering and retrieval operations.

Column	Purpose
created_at	Time-series queries and trend analysis
site_id	Site-level filtering
rig_id	Rig-level filtering
device_id	Device history and device-specific searches
event	Event analysis and diagnostics
hard_kill	Safety event investigation

Table 10. Database indexing strategy.

6.6. Data Retention Policy

A structured data-retention policy is recommended to balance long-term analytical value with database storage requirements. Raw telemetry data are retained for operational monitoring, model training, and fault analysis, while summary records and critical system events are preserved for long-term historical reference.

The recommended retention policy is summarized in Table 11.

Data Type	Retention
Raw telemetry	30 to 90 days
Events and alarms	1 year
Pump-cycle summaries	Long-term retention
ML predictions and MPC recommendations	90 days
MQTT message logs	7 to 30 days
Firmware update events	Long-term retention

Table 11. Recommended database retention policy.

To ensure data integrity and disaster recovery capability, database backups should be performed on a regular schedule using PostgreSQL backup utilities. For production deployments, daily database backups and weekly archived snapshots are recommended. Additional off-site backup storage may be implemented for enhanced system resilience and long-term record preservation.

7. Control System Design

7.1. Control Objectives

The SCC platform is designed to regulate solution temperature during stress-corrosion-cracking (SCC) testing while maintaining safe and stable operation during heating, fluid circulation, and long-duration experiments. The control system combines closed-loop temperature regulation, pump-aware compensation, and firmware-level safety mechanisms to ensure reliable operation under varying thermal conditions.

The primary control objectives are summarized in **Table 12**.

Objective	Description
Maintain setpoint temperature	Control the heater output to reach and hold the target bath temperature.
Prevent overheating	Disable heating when the temperature approaches unsafe limits.
Minimize thermal oscillation	Use PID control and ramped setpoints to reduce overshoot and unstable cycling.
Compensate for pump disturbance	Add heater bias before and during pump operation to reduce temperature drop.
Ensure safe operation	Apply heater lockout, hard-kill logic, and runtime limits in firmware.

Table 12. Control objectives implemented within the SCC platform.

7.2. Current Control Architecture

The primary control loop is executed locally within the Arduino controller firmware. Temperature measurements are acquired through the NTC thermistor and used to generate heater PWM commands. The ESP32 functions as a communication gateway, forwarding telemetry and command messages between the controller and backend infrastructure. Higher-level functions, including data storage, alarm generation, machine-learning prediction, and advisory model predictive control (MPC), are executed within the backend system.

The current control architecture is summarized in Table 13.

Control Feature	Implementation
Main controller	Arduino firmware
Communication bridge	ESP32 relay
Control output	Heater PWM

Feedback input	Temperature sensor through ADC
Control interval	700 ms
Final setpoint	125.00 °C
Heater PWM maximum	200
Hard-kill temperature	130.00 °C
Heater lockout high threshold	128.50 °C
Heater lockout reset threshold	126.50 °C

Table 13. SCC control system configuration.

This architecture ensures that critical control and safety functions remain operational even if communication with the backend is interrupted.

7.3. Control Modes

The controller operates as a finite-state system consisting of multiple control modes that support autotuning, temperature regulation, and safety management. During startup, the controller performs relay-based autotuning to estimate PID parameters. Once tuning is completed, the controller transitions to PID-based operation and maintains the target process temperature.

The available control modes are summarized in Table 14.

Mode	Description
Autotune warmup	Heater warms the system until it reaches the autotune region.
Autotune relay	Relay cycling is used to estimate PID tuning parameters.
PID ramp	The active setpoint ramps gradually from the autotune target to the final setpoint.
PID hold	The controller holds the final setpoint using PID control.
Safety override	Heating is disabled during hard kill, manual kill, or heater lockout conditions.

Table 14. Controller operating modes.

The controller automatically transitions between modes based on measured temperature, tuning completion status, and safety conditions.

7.4. Safety Logic

Safety functions are implemented directly within the controller firmware to ensure protection independent of the database, dashboard, network connection, machine-learning models, or backend services. Firmware-level protection guarantees that unsafe operating conditions result in immediate corrective action regardless of external system status.

The implemented safety mechanisms are summarized in Table 15.

Safety Function	Trigger	Action
Heater lockout	Temperature ≥ 128.50 °C	Heater output disabled
Lockout reset	Temperature ≤ 126.50 °C	Heater may resume
Hard kill	Temperature ≥ 130.00 °C	Heater and pump are disabled
Manual kill	Operator or external safety input	Control recommendation forced to zero
Runtime limit	96 hours	Heater and pump disabled

Table 15. Firmware safety functions.

The machine-learning and MPC systems operate exclusively in an advisory capacity and shall not be considered part of the primary safety layer.

7.5. Pump Aware Compensation.

Pump operation introduces a predictable thermal disturbance by circulating cooler fluid through the test system. To reduce temperature excursions caused by pumping events, the controller applies anticipatory heater bias both before and during pump operation.

The pump compensation parameters are summarized in Table 16.

Parameter	Value	Description
Pump period	30000 ms	Pump cycle interval
Pump on time	1000 ms	Pump active duration
Pre-bias window	5000 ms	Heater bias starts before pump activation
Pre-bias PWM	+18	Extra heater PWM before pump operation
Pump-on bias PWM	+28	Extra heater PWM while pump is running

Table 16. Pump-aware compensation settings.

To reduce integral windup caused by predictable disturbances, the PID integrator is temporarily frozen while the pump is active.

7.6. Advisory Model Predictive Control (MPC)

An advisory Model Predictive Control (MPC) module is implemented within the backend architecture to evaluate future control actions and provide operator guidance. The MPC module does not directly control the heater; instead, it evaluates candidate heater PWM values and recommends an optimal action based on predicted thermal response.

Current MPC Status: Advisory Only

Future MPC Status: Closed-Loop Control Following Validation and Safety Review

The current MPC implementation is summarized in Table 17.

MPC Feature	Description
Candidate PWM values	0, 64, 128, 184, 220, 255
Prediction horizon	5 seconds
Model source	`best_model_5s.joblib` if available
Fallback behavior	Uses a simple predictor when no ML model exists
Safety override	Recommends PWM 0 during hard kill, manual kill, or heater lockout

Table 17. Advisory MPC configuration

The MPC cost function minimizes temperature error while penalizing excessive control effort and abrupt changes in heater output:

$$cost = (predicted_{temp} - setpoint_c)^2 + 0.0005 * pwm^2 + 0.05 * (pwm - previous_{pwm})^2$$

7.7. Machine Learning Roadmap

Model source: best_model_5s.joblib if available, generated using Scikit-learn and XGBoost model pipelines [11,12]. ML models provide temperature forecasting and anomaly detection capabilities that support process monitoring and future control-system enhancements. All ML outputs are advisory and do not override firmware-level safety protections.

Planned ML applications are summarized in Table 18.

Application	Purpose
Process Prediction	Forecast future process temperatures over short time horizons
Fault Detection	Identify abnormal heater, pump, or sensor behavior
Anomaly Detection	Detect unusual telemetry patterns during operation
Predictive Maintenance	Estimate equipment degradation using historical trends

Application	Purpose
MPC Support	Provide future temperature predictions for advisory MPC calculations

Table 18. Machine-learning applications within the SCC platform.

Current prediction horizons include short-term forecasts of +1 s, +5 s, and +10 s when trained models are available. Future development will focus on model validation, uncertainty quantification, adaptive control integration, and eventual deployment of closed-loop predictive control strategies.

8. Safety Analysis

The SCC control platform is designed so safety-critical decisions remain on the hardware side of the system. The Arduino controller and electrical interlocks own heater and pump shutdown behavior. The ESP32, Flask backend, database, HMI, ML model, and MPC service provide communication, monitoring, logging, alarms, and advisory recommendations only.

Backend alarms and dashboard notifications improve operator visibility, but they are not the primary protection layer. The rig must remain safe if the network, MQTT broker, database, server, HMI, ML model, or MPC endpoint fails.

8.1. Hazards

A hazard assessment was performed to identify potential failure modes, their consequences, and existing mitigation measures. The identified hazards are summarized in Table 19.

Hazard	Potential Consequence	Existing Mitigation
Bath overheating	Equipment damage, accelerated corrosion, unsafe test conditions, or operator burn hazard	Firmware heater lockout at 128.5 °C, hard kill at 130.0 °C, heater PWM limiting, and backend high-temperature alarms
Thermistor failure or invalid ADC reading	Incorrect temperature feedback resulting in uncontrolled heating	Sensor fault detection, ADC validation checks, heater shutdown, pump shutdown, lockout activation, and alarm generation
Pump failure or blocked flow	Poor solution circulation, invalid test results, and delayed thermal recovery	Pump telemetry monitoring, recovery-time analysis, and backend alarm generation
Pump operation without sufficient thermal headroom	Excessive temperature drop below operating conditions	Temperature-gated pump operation with configurable upper and lower thresholds
Heater output failure or	Thermal runaway and equipment damage	Firmware lockout logic, hard-kill protection, PWM limiting, and

Hazard	Potential Consequence	Existing Mitigation
excessive PWM command		recommended independent power isolation
Manual emergency shutdown	Delayed operator response during abnormal conditions	Manual kill command and provision for physical emergency-stop hardware
Network, MQTT, server, database, or HMI failure	Loss of monitoring, alarms, and remote control capability	Local firmware continues operation independently of backend services
ML or MPC recommendation error	Unsafe recommendation if treated as direct control action	MPC remains advisory only and cannot override firmware safety logic
Firmware update interruption	Uncertain controller state during firmware update operations	OTA preparation disables heater and motor outputs before reset
Excessive runtime or unattended operation	Continued operation beyond intended test duration	Firmware runtime limit and automatic shutdown functionality

Table 19. Hazard analysis and mitigation measures.

8.2. Safety Features

Multiple independent safety mechanisms are implemented within the SCC platform to prevent unsafe operation and reduce risk associated with hardware faults, software faults, and operator error.

The primary safety functions are summarized below:

- **Heater Lockout:** Heater output is disabled when measured temperature reaches 128.5 °C. Heating may resume only after temperature decreases below 126.5 °C.
- **Hard Kill Protection:** At 130.0 °C, the controller immediately disables heater and pump outputs, clears control commands, resets the PID integrator, activates lockout status, and generates a hard-kill event.
- **Manual Kill Function:** Operator-issued kill commands immediately disable heater and pump outputs and prevent further actuation until reset.

- **Sensor Fault Detection:** Invalid ADC values and thermistor fault conditions automatically disable all outputs and generate a fault event.
- **Safe Startup Sequence:** Heater and motor outputs remain disabled during initialization until controller state validation is complete.
- **Pump Temperature Gating:** Pump operation is permitted only within an approved temperature operating window.
- **PWM Limiting:** Heater PWM commands are constrained to a configurable maximum value to prevent excessive power output.
- **PID Windup Protection:** Integral accumulation is reset during lockout, hard-kill, manual-kill, and sensor-fault conditions.
- **Safe OTA Preparation:** Firmware update procedures disable all outputs before initiating reset or programming operations.
- **Alarm Logging and Audit Trail:** Critical safety events are recorded within the database and displayed through the HMI.
- **Advisory-Only MPC:** Model-based recommendations are prevented from overriding firmware safety states

8.3. Emergency Shutdown Procedure

In the event of abnormal operation or a hazardous condition, the following emergency shutdown procedure shall be performed:

1. Emergency shutdown order should issue a manual kill before disconnecting power only if HMI is available; otherwise, disconnect power first.
2. Disconnect the power
3. Verify that heater PWM output is zero and heater power is electrically isolated.
4. Verify that all pump and motor outputs are disabled.
5. Allow the bath temperature to decrease below the lockout reset threshold.
6. Record the event, including timestamp, measured temperature, active alarms, and operator actions.
7. Inspect the heater assembly, thermistor, pump system, power electronics, and enclosure for damage or abnormal conditions.
8. Restart operation only after the root cause has been identified and corrective actions have been completed.

8.4. Safety verification Tests

Safety functions shall be validated through functional testing before deployment. Verification tests are summarized in Table 20.

Safety Function	Test Method	Expected Result
Safe Startup	Power cycle controller while monitoring outputs	Outputs remain disabled during initialization
Heater Lockout	Raise temperature to 128.5 °C	Heater output disabled and lockout activated
Lockout Recovery	Reduce temperature below 126.5 °C	Lockout clears and normal operation resumes
Hard Kill	Simulate temperature ≥ 130.0 °C	Heater and pump disabled; hard-kill event generated
Sensor Fault Detection	Simulate open-circuit or short-circuit thermistor	Outputs disabled and fault event generated
Manual Kill	Issue kill command through communication interface	Heater and pump immediately disabled
Pump Temperature Gating	Test operation at threshold temperatures	Pump operates only within permitted temperature range
Runtime Limit	Simulate 96-hour runtime condition	Heater and pump automatically disabled
Backend Alarm Path	Inject alarm-triggering telemetry data	Correct alarm generation and logging
MPC Safety Override	Request recommendation during safety event	Recommended PWM returns zero

Table 20. Safety verification tests.

8.5. Residual Risk and Recommendation

Although multiple layers of protection are implemented, certain residual risks remain and should be addressed prior to unattended or production operation.

- Installation of a physical emergency-stop device or independent power disconnect is strongly recommended.
- Thermistor calibration and fault-detection thresholds should be validated experimentally using open-circuit, short-circuit, and elevated-temperature conditions.

- Heater MOSFET, relay, or power-stage failures may bypass software control; independent over-temperature protection and fused power isolation should be considered.
- Network failures eliminate remote visibility and alarm reporting; local indicators and periodic operator inspection remain necessary.
- OTA firmware update functionality should remain restricted until complete hardware validation has been performed.
- Any future implementation of closed-loop MPC control shall require a dedicated safety review, cybersecurity review, and verification that firmware safety mechanisms cannot be bypassed.

The safety architecture implemented within the SCC platform ensures that hazardous conditions are mitigated through local firmware protection mechanisms while maintaining operational visibility through backend monitoring, alarm management, and data logging services.

9. Verification and Validation

9.1. Verification and Validation Overview

Verification and validation activities were developed to evaluate the functionality, reliability, and safety of the SCC platform. Verification focuses on confirming that individual software modules, firmware functions, communication interfaces, and safety mechanisms perform according to their design requirements. Validation focuses on demonstrating that integrated subsystems operate correctly when deployed as a complete platform.

The SCC platform repository currently contains automated unit tests covering firmware logic, ESP32 relay parsing, backend telemetry processing, alarm generation, firmware management functions, API authentication, and advisory MPC safety behavior. In addition, the repository includes manual integration procedures for telemetry-path validation and dedicated hardware test projects for thermistor, heater, and motor verification.

The verification matrices presented in this section include only tests and validation procedures that currently exist within the repository. Planned activities that have not yet been implemented, including dashboard automation testing, heartbeat monitoring validation, long-duration endurance testing, and full commissioning activities, are excluded from the formal verification record and are identified separately as future work.

9.2. Unit Tests

Unit testing is used to verify individual firmware and software functions in isolation before subsystem integration. These tests evaluate temperature measurement, safety logic, pump control, telemetry generation, communication parsing, backend processing, firmware management, and MPC advisory functions.

The implemented unit test matrix is summarized in Table 21.

Test ID	Description	Expected Result	Status
UT-001	Temperature conversion and ADC calibration	ADC values are converted to expected temperature values using the calibrated transfer function	Implemented
UT-002	ADC sensor fault detection	Open-circuit and short-circuit ADC rail values are detected with recovery hysteresis	Implemented

Test ID	Description	Expected Result	Status
UT-003	Pump temperature gating	Pump permission enables only with thermal headroom and disables at low or high temperature limits	Implemented
UT-004	Pump PWM timing	Pump startup kick, run PWM, and off timing follow the configured motor cycle	Implemented
UT-005	Motor timer and heater pre-bias logic	Motor cycle phase and heater bias windows are calculated correctly	Implemented
UT-006	Autotune and PID state logic	Autotune transitions, timeout behavior, PID first-sample behavior, ramp completion, and lockout output suppression work as expected	Implemented
UT-007	Telemetry CSV schema	Telemetry header column count and column order match the expected schema	Implemented
UT-008	Bootloader entry command parsing	Supported bootloader preparation commands are accepted and unrelated commands are rejected	Implemented
UT-009	ESP32 relay CSV parsing	Arduino CSV telemetry rows are parsed into the relay snapshot fields correctly	Implemented
UT-010	ESP32 relay backend command parsing	Supported backend command types are parsed and unsupported commands are rejected	Implemented
UT-011	Backend telemetry parser	Headerless and header-based CSV telemetry rows are parsed and invalid rows are rejected	Implemented

Test ID	Description	Expected Result	Status
UT-012	Backend alarm rules	Hard kill, manual kill, heater lockout, sensor fault, high temperature, slow recovery, and excessive cooling alarms are generated correctly	Implemented
UT-013	Backend API authentication	Protected telemetry write endpoints reject missing keys and accept valid API key formats	Implemented
UT-014	Firmware API validation and command queue	ESP32 and Arduino firmware file types are validated and queued commands can be acknowledged	Implemented
UT-015	MPC safety override	MPC recommendations return zero PWM during hard kill, manual kill, or heater lockout states	Implemented

Table 21. Implemented unit tests for SCC firmware, ESP32 relay services, backend services, and MPC logic.

Successful execution of these tests verifies that the individual software and firmware components operate according to their intended design requirements and interface specifications.

9.3. Integration and Hardware Validation Procedures

Integration testing verifies communication and interoperability between embedded controllers, communication services, backend infrastructure, and database storage systems. Hardware validation procedures are used to confirm operation of physical components, including temperature sensing, heater control, and pump actuation.

The integration and hardware validation procedures currently implemented within the repository are summarized in Table 22.

Test ID	Description	Expected Result	Status
IT-001	Arduino telemetry to ESP32 relay parsing	ESP32 receives deterministic Arduino CSV rows and parses them with the relay core parser	Manual procedure defined

Test ID	Description	Expected Result	Status
IT-002	ESP32 telemetry relay to Flask API	ESP32 posts parsed telemetry JSON to the Flask telemetry endpoint	Manual procedure defined
IT-003	Flask telemetry ingestion to PostgreSQL	Stored PostgreSQL rows match the source Arduino CSV fixture	Manual verifier implemented
IT-004	End-to-end telemetry pipeline	Data propagate from Arduino test firmware through ESP32 and Flask into PostgreSQL	Manual procedure defined
IT-005	Backend-only telemetry pipeline smoke test	Sample CSV rows are posted directly to Flask and verified against PostgreSQL without hardware	Scripted verifier implemented
IT-006	Standalone NTC temperature readout	Arduino reads thermistor ADC values and prints live temperature CSV rows	Manual hardware test project defined
IT-007	Standalone heater control test	Arduino heater test accepts start/stop commands, limits PWM, and reports hard-kill status	Manual hardware test project defined
IT-008	Standalone motor output test	Arduino motor test accepts start/stop commands and reports PWM output state	Manual hardware test project defined

Table 22. Implemented integration tests and hardware validation procedures.

These procedures provide verification of data flow through the telemetry pipeline, communication between hardware and software subsystems, database storage integrity, and operation of critical hardware components. Together, they establish confidence that the individual subsystems can function as an integrated SCC platform.

9.4. Current Verification Coverage Gaps

Although significant verification coverage has been implemented, several system-level validation activities remain outside the current repository scope. These items represent future

verification work that should be completed before deployment in a production or long-duration testing environment.

The repository does not currently include automated verification for dashboard live-update functionality, MQTT broker transport validation, heartbeat and offline-device detection, complete HMI-to-controller manual-kill routing, or full system commissioning and endurance testing. These activities should remain classified as future validation tasks until formal test procedures, execution records, and acceptance criteria have been developed.

Future verification efforts should also include extended-duration operational testing, failure-recovery testing, network-loss testing, and final acceptance testing of the fully integrated SCC platform.

9.5. Verification Summary

The SCC platform currently includes implemented verification activities covering firmware control logic, safety mechanisms, telemetry generation, communication parsing, backend processing, firmware management, and advisory MPC behavior. Additional integration procedures and standalone hardware validation projects have been developed to support verification of communication pathways and physical hardware operation.

The existing verification framework provides a foundation for future system-level validation and commissioning activities while demonstrating that the major software, firmware, and hardware subsystems have defined verification procedures and measurable acceptance criteria.

10. Deployment Guide

10.1. Deployment Overview

The SCC platform is deployed using a containerized architecture based on Docker and Docker Compose [6] consisting of a Flask backend, React Human–Machine Interface (HMI), PostgreSQL database, MQTT broker, and optional Cloudflare Tunnel service. Docker Compose is used to simplify deployment and ensure consistent configuration across development and production environments.

The deployment architecture supports local operation on a single workstation or distributed deployment across multiple networked systems. The ESP32 communication gateway publishes telemetry data through MQTT, while backend services process, store, and visualize operational data.

10.2. Server Requirements

The deployment server must provide sufficient computing resources to support the backend API, database services, MQTT communication, dashboard visualization, and future machine-learning workloads.

Operating System: Ubuntu Server Lite

Processor: Intel NUC

Memory: 4 GB RAM

Storage: 128GB

Network: The deployment server must be reachable by the ESP32 communication gateway.

Required ports include:

Service	Port
MQTT Broker	1883
Backend API	5050
Frontend Dashboard	5173
PostgreSQL	5432

Table 23. Network Ports

Software Requirements:

- Docker Engine or Docker Desktop
- Docker Compose
- Python 3.11 (non-Docker deployment)

- Node.js 22 (non-Docker deployment)
- PostgreSQL
- Eclipse Mosquitto MQTT Broker

10.3. Containerized Services

The SCC platform is composed of multiple Docker services that provide communication, data storage, visualization, firmware management, and predictive analytics capabilities.

Service	Image or Build Source	Port	Purpose
Backend	backend/Dockerfile using Python 3.11-slim	5050:5000	Runs the Flask API, MQTT subscriber, telemetry parser, alarm engine, ML prediction services, MPC advisory logic, and firmware management APIs
Frontend	node:22-alpine	5173:5173	Provides the React/Vite HMI dashboard for monitoring, alarms, trends, settings, firmware management, and analytics
PostgreSQL	postgres:16	5432:5432	Stores telemetry, alarms, events, pump-cycle records, predictions, commands, and configuration data
MQTT Broker	eclipse-mosquitto:2	1883:1883, 9001:9001	Provides MQTT and MQTT-over-WebSocket communication services
Cloudflare Tunnel	cloudflare/cloudflared:latest	Outbound Tunnel	Optional secure remote-access service

Table 24. Docker services used within the SCC platform deployment.

10.4. Deployment Procedure

The following procedure is used to deploy the SCC platform using Docker Compose.

Step 1 – Obtain the Source Repository

```
git clone <repository-url>
cd scc-control-platform
```

Step 2 – Create the Environment Configuration File

```
cp .env.example .env
```

Step 3 – Configure Environment Variables

The minimum required configuration is shown below.

```
API_WRITE_KEY=<shared-write-key-for-backend-and-dashboard>
CORS_ORIGINS=http://localhost:5173
PUBLIC_BASE_URL=http://localhost:5050
MQTT_HOST=mosquitto
MQTT_PORT=1883
MQTT_BASE_TOPIC=scc
```

If email notifications are enabled, SMTP configuration parameters must also be defined.

Step 4 – Start Platform Services

```
docker compose up --build
```

Step 5 – Verify Service Status

```
docker compose ps
```

Step 6 – Verify Application Access

```
Backend API: http://localhost:5050
Frontend HMI: http://localhost:5173
PostgreSQL: localhost:5432
MQTT Broker: localhost:1883
```

Step 7 – Configure ESP32 Communication Gateway

Update the ESP32 firmware configuration to match the deployment environment.

```
#define MQTT_BROKER_URI "mqtt://<server-lan-ip>:1883"
#define MQTT_BASE_TOPIC "scc"
#define MQTT_SITE_ID "site-01"
#define MQTT_RIG_ID "rig-01"
#define MQTT_DEVICE_ID "esp32-relay-01"
```

Step 8 – Verify Telemetry Communication

Confirm that telemetry messages are received by the backend and appear within the dashboard.

```
GET http://localhost:5050/api/latest
```

Step 9 – Execute Verification Procedures

Prior to operation, execute available automated verification tests.

```
pytest backend/tests
pio test -d firmware/arduino/SCC-V1.4 -e native
pio test -d firmware/esp32/ESP32-Relay -e native
```

Step 10 – Configure Optional Remote Access

For remote access deployments, configure the Cloudflare Tunnel token to enable secure external access without direct port exposure [13] within the environment configuration file.

```
CLOUDFLARE_TUNNEL_TOKEN=<cloudflare-token>
```

After deployment, verify that public endpoints correctly route traffic to the backend API and HMI dashboard.

10.5. Post-Deployment Verification

Following deployment, a series of checks should be performed to confirm proper system operation.

Check	Expected Result
Backend Health	Backend responds successfully on http://localhost:5050
Dashboard Access	HMI dashboard loads on http://localhost:5173
Database Startup	PostgreSQL container reports healthy status
MQTT Connectivity	ESP32 successfully connects to the MQTT broker
Telemetry Reception	/api/latest returns current telemetry data
Alarm Functionality	Active alarms appear within API responses and dashboard views
Manual Kill Command	Authenticated kill commands can be issued successfully
Data Persistence	PostgreSQL data remain available following container restart

Table 25. Post-deployment verification checklist.

Successful completion of the post-deployment verification checklist indicates that the SCC platform has been deployed correctly and is ready for hardware integration, system validation, and operational testing.

11. Operations Guide

11.1. Startup Procedure

Before operation, the SCC platform shall be inspected to verify that all mechanical, electrical, thermal, and communication systems are functioning correctly. The startup procedure ensures that the specimen, heating cell, circulation system, sensors, and control electronics are prepared for safe operation before heating is enabled.

The startup procedure shall be performed in the order listed below.

1. Wear personal protective equipment (PPE): Gloves and Safety glasses.
2. Mount the test specimen securely within the test fixture.
3. Apply RTV sealant around the copper cell sealing surfaces as required.
4. Install and secure the copper heating cell.
5. Connect the pump outlet tubing to the copper cell.
6. Connect the exhaust tubing to the copper cell.
7. Verify that the MgCl_2 reservoir contains sufficient solution for the planned test duration.
8. Verify that the pump intake tubing remains fully submerged below the fluid surface.
9. Verify thermistor and temperature sensor connections.
10. Verify heater wiring and heater cartridge installation.
11. Verify all control electronics, power connections, and communication cables.
12. Apply power to the SCC platform.
13. Verify network connectivity between the ESP32 gateway and backend services.
14. Allow the control system to complete initialization and startup checks.
15. Verify dashboard status indicators and confirm receipt of live telemetry.

Successful completion of the startup procedure indicates that the SCC platform is ready for operation.

11.2. Corrosive Medium Preparation and Handling

The SCC platform utilizes a magnesium chloride (MgCl_2) solution as the corrosive medium for stress-corrosion-cracking experiments. Proper preparation of the solution is required to ensure repeatable test conditions and consistent corrosion behavior.

Solution Composition

The standard corrosion solution used for SCC testing is summarized in Table 26.

Parameter	Value
Chemical	Magnesium Chloride Hexahydrate ($\text{MgCl}_2 \cdot 6\text{H}_2\text{O}$)
Solution Concentration	30 wt.% MgCl_2
$\text{MgCl}_2 \cdot 6\text{H}_2\text{O}$ Mass	500 g

Parameter	Value
Deionized Water Volume	280 mL
Typical Operating Temperature	125 °C

Table 26. Standard MgCl₂ corrosion solution.

Preparation Procedure

1. Wear personal protective equipment (PPE): ideally, safety glasses, gloves, and a mask.
2. Verify that a clean mixing container is available.
3. Measure 280 mL of deionized (DI) water.
4. Add 500 g of magnesium chloride hexahydrate (MgCl₂·6H₂O) to the container.
5. Slowly add the DI water while stirring continuously.
6. Continue mixing until all visible solids are dissolved and a uniform solution is obtained.
7. Allow the solution to stabilize before transferring it to the SCC reservoir.
8. Verify that the reservoir contains sufficient solution volume before beginning operation.

11.3. Operator Responsibilities and Monitoring Requirements

During operation, the operator is responsible for maintaining safe operating conditions, monitoring system performance, and responding to alarms or abnormal conditions.

Operator Responsibilities

- Confirm that the SCC rig is physically safe to operate before enabling heat or pump activity.
- Keep the HMI dashboard open throughout testing and verify that telemetry updates continuously.
- Monitor bath temperature, temperature error, heater PWM, pump status, and controller operating mode.
- Verify that the system remains within the intended operating temperature range.
- Confirm that heater lockout, manual kill, and hard-kill indicators remain inactive during normal operation.
- Verify that pump cycles occur only when sufficient thermal headroom is available.
- Review active alarms and event timeline entries throughout each operating session.
- Treat machine-learning predictions and MPC recommendations as advisory information only.
- Record abnormal behavior, alarms, operator actions, and shutdown events in the test log.
- Immediately stop testing if unsafe operating conditions are observed.

Monitoring Requirements

The following parameters shall be monitored during active operation:

- Process temperature, setpoint temperature, and temperature error.
- Heater PWM command and heater operating state.
- Pump status, motor PWM, pump recovery time, and pump-cycle temperature drop.
- Active alarm status and alarm history.
- Event timeline entries, including pump start, pump stop, pump recovery, hard-kill, and sensor-fault events.
- MQTT connectivity and dashboard update status.
- Database logging status and telemetry history availability.

Operators shall remain in the vicinity of the test system during active heating unless approved unattended-operation procedures and independent physical safety systems are in place.

Any abnormal noise, fluid leakage, overheating, unexpected pump behavior, unusual odor, electrical fault indication, or loss of telemetry shall require immediate investigation and possible shutdown.

11.4. Alarm Response Procedures

The SCC platform generates alarms to notify operators of abnormal operating conditions. Critical alarms require immediate corrective action, while warning alarms require increased monitoring and investigation.

Alarm	Severity	Operator Response
HARD_KILL	Critical	Treat as an emergency shutdown. Verify heater PWM is zero, verify pump output is disabled, allow cooling, inspect hardware, and determine root cause before restart.
MANUAL_KILL	Critical	Confirm shutdown was intentional. Maintain disabled state until inspection is complete and restart conditions are met.
SENSOR_FAULT	Critical	Stop operation, inspect thermistor wiring and ADC input, verify temperature independently if possible, and restart only after valid telemetry is restored.
HIGH_TEMP_CRITICAL	Critical	Disable heating immediately, allow cooling, and inspect heater and temperature-control hardware.

Alarm	Severity	Operator Response
HEATER_LOCKOUT	Warning	Continue monitoring while heating remains disabled. Resume operation only after temperature falls below the reset threshold.
HIGH_TEMP_WARNING	Warning	Monitor temperature trend and heater output closely. Prepare for shutdown if temperature continues increasing.
SLOW_RECOVERY	Warning	Inspect pump operation, fluid flow path, mixing performance, and thermal recovery characteristics.
EXCESSIVE_PUMP_COOLING	Warning	Verify pump timing, flow rate, solution temperature, and compensation settings.

Table 27. Alarm response procedures.

If any critical alarm occurs, operator safety and equipment protection shall take priority over test completion or data collection. Testing shall remain suspended until the root cause has been identified and corrective action has been completed.

11.5. Shutdown Procedure

The SCC platform shall be shut down in a controlled manner to prevent equipment damage, fluid spills, and unnecessary exposure to heated MgCl_2 solution.

The shutdown procedure shall be performed in the order listed below.

1. Wear all required personal protective equipment (PPE).
2. Issue the manual kill command through the HMI or controller interface.
3. Disconnect power from the SCC platform.
4. Disconnect the intake tubing from the pump and allow residual solution to drain.
5. Disconnect the pump outlet tubing from the copper cell and return residual solution to the reservoir.
6. Remove the copper cell assembly from the specimen.
7. Remove the test specimen from the fixture.
8. Clean the copper cell, tubing, specimen, and work area.
9. Inspect the system for corrosion, leaks, damaged wiring, or abnormal wear.
10. Record test completion details, observations, alarms, and maintenance actions in the test log.

Successful completion of the shutdown procedure returns the SCC platform to a safe, de-energized condition suitable for storage, maintenance, or preparation for the next test.

12. Maintenance Manual

12.1. Maintenance Overview

Routine maintenance is required to ensure reliable operation of the SCC platform and to minimize the risk of equipment failure, inaccurate measurements, communication interruptions, and unsafe operating conditions. Maintenance activities are divided into preventive maintenance and corrective maintenance. Preventive maintenance consists of scheduled inspections and verification activities intended to identify potential issues before failure occurs. Corrective maintenance is performed in response to alarms, faults, abnormal system behavior, or equipment malfunction.

All maintenance activities should be recorded in the system maintenance log, including inspection dates, observations, corrective actions, replacement parts, and personnel performing the work.

12.2. Preventive Maintenance

Preventive maintenance shall be performed according to the schedule shown in Table 27. These activities are intended to verify proper operation of sensing, heating, pumping, communication, and data-management systems before faults occur.

Item	Frequency
Temperature sensor inspection	Daily
Heater and pump status check	Daily
SCC test cell inspection	Daily
Wiring and connector inspection	Weekly
MQTT/backend connectivity check	Weekly
Database backup verification	Monthly
Firmware and configuration backup	Monthly
Alarm and manual kill function check	Monthly

Table 28. Preventive maintenance schedule.

Daily Inspection Requirements

- Inspect the thermistor and sensor wiring for damage, corrosion, or loose connections.
- Verify proper heater operation and confirm normal heating response.
- Verify pump operation and check for leaks, blockage, or abnormal noise.
- Inspect the SCC test cell for corrosion, contamination, fluid leakage, or physical damage.
- Verify that telemetry data are being reported correctly through the dashboard.

Weekly Inspection Requirements

The following items should be verified at least once per week:

- Inspect all electrical wiring, connectors, and terminal blocks.
- Verify MQTT communication between the ESP32 relay and backend services.
- Review alarm history for recurring faults or abnormal trends.
- Inspect tubing and fluid pathways for signs of wear, clogging, or degradation.

Monthly Inspection Requirements

The following items should be verified monthly:

- Confirm successful PostgreSQL database backups.
- Create backups of firmware images and system configuration files.
- Verify operation of manual kill, heater lockout, and hard-kill safety functions.
- Review system logs for persistent alarms, communication issues, or abnormal operating conditions.

12.3. Corrective Maintenance

Corrective maintenance shall be performed whenever a fault, alarm condition, communication failure, hardware malfunction, or abnormal operating condition is observed. The objective of corrective maintenance is to identify the root cause of the issue, restore normal operation, and prevent recurrence.

The recommended corrective actions for common fault conditions are summarized in Table 28.

Fault Condition	Corrective Action
Sensor fault	Inspect thermistor wiring, verify ADC readings, replace sensor if needed
Heater not responding	Check heater wiring, MOSFET/relay, power supply, PWM output, and firmware state
Pump not operating	Inspect pump wiring, motor driver, tubing, flow path, and motor PWM output
Excessive temperature	Stop operation, verify heater output is off, inspect sensor accuracy and heater control circuit

Fault Condition	Corrective Action
Telemetry loss	Check ESP32 power, Wi-Fi/MQTT connection, backend service, and broker status
Database logging failure	Check PostgreSQL container, disk space, backend logs, and database connection string
Dashboard not updating	Check backend API, frontend service, network connection, and browser console errors

Table 29. Corrective maintenance procedures.

12.4. Component Replacement Guidelines

When replacing components, power shall be removed from the system, and all heated surfaces shall be allowed to cool to a safe handling temperature. Replacement components shall meet the original design specifications and shall be verified through functional testing before returning the system to service.

The following components are considered field-replaceable:

- NTC thermistor temperature sensor
- Cartridge heater
- Pump and pump tubing
- ESP32 communication module
- Wiring harnesses and electrical connectors
- Power switching components, including relays and MOSFETs

Following replacement, a startup inspection and verification test shall be performed to confirm correct operation.

12.5. Maintenance Safety Considerations

Maintenance activities shall be performed only when the system is in a safe, de-energized condition. Operators shall wear appropriate personal protective equipment when handling heated components, corrosive solutions, electrical systems, or contaminated hardware.

Before performing maintenance:

1. Issue a manual kill command if the system is operational.
2. Disconnect system power.
3. Allow the copper heating cell and associated hardware to cool.
4. Drain or isolate MgCl_2 solution if fluid-handling components will be serviced.
5. Verify that all heater and pump outputs are de-energized.
6. Follow applicable laboratory safety procedures for chemical handling and disposal.

13. Troubleshooting Guide

13.1. Troubleshooting Overview

This section guides diagnosing and correcting common hardware, firmware, communication, database, and operational issues that may occur during SCC platform operation. Troubleshooting should be performed systematically, beginning with verification of safety conditions, power availability, sensor status, and communication connectivity before attempting more advanced diagnostics.

When investigating faults, operators should review active alarms, telemetry values, event logs, backend logs, and dashboard status indicators. If a critical alarm such as `HARD_KILL`, `MANUAL_KILL`, or `SENSOR_FAULT` is active, the system shall remain in a safe state until the root cause has been identified and corrected.

The troubleshooting matrix is summarized in Table 29

Issue	Possible Cause	Corrective Action
No temperature data	Thermistor disconnected, ADC input fault, Arduino not running, or ESP32 not forwarding telemetry	Inspect thermistor wiring, verify Arduino serial output, check ESP32 power, and confirm telemetry reaches the backend
Temperature value is unrealistic	Sensor calibration error, loose thermistor connection, ADC rail reading, or damaged sensor	Check ADC value, inspect wiring, compare with an independent thermometer, replace sensor if needed, and recalibrate
<code>SENSOR_FAULT</code> alarm active	Open-circuit or short-circuit sensor condition, missing temperature field, or invalid ADC data	Stop operation, inspect sensor circuit, restore valid readings, and restart only after the alarm clears
No MQTT data	Mosquitto broker offline, wrong broker IP, Wi-Fi issue, wrong	Verify broker container is running, confirm ESP32 uses the server LAN IP, check Wi-Fi credentials, and confirm topic format

Issue	Possible Cause	Corrective Action
	MQTT topic, or ESP32 disconnected	
Backend not receiving telemetry	MQTT subscriber offline, backend container stopped, bad API key for HTTP test path, or malformed payload	Check backend logs, restart backend if needed, verify MQTT settings, and validate telemetry format
Dashboard not updating	Frontend cannot reach backend, backend API down, stale browser session, or telemetry not arriving	Open <code>http://localhost:5050/api/latest</code> , check frontend API URL, refresh the dashboard, and verify backend/container status
PostgreSQL logging failure	Database container unhealthy, wrong <code>DATABASE_URL</code> , schema not initialized, or disk full	Check <code>docker compose ps</code> , inspect database logs, verify connection string, confirm free disk space, and restart services if required
Heater disabled	Heater lockout, hard kill, manual kill, sensor fault, temperature above setpoint, or runtime limit reached	Check active alarms and telemetry flags, verify temperature limits, clear manual kill only after inspection, and allow cooling before restart
Heater does not heat when expected	PWM output disabled, MOSFET/relay fault, power supply issue, wiring fault, or failed heater	Verify heater PWM telemetry, inspect power electronics, measure output voltage safely, and test heater circuit before operation
Temperature rising too fast	Heater control fault, stuck MOSFET/relay, sensor reading low, or excessive heater power	Stop the test, verify heater output is de-energized, inspect sensor accuracy, and check heater driver hardware

Issue	Possible Cause	Corrective Action
Pump not running	Pump temperature gate disabled, motor driver fault, wiring fault, pump jam, manual kill, hard kill, or runtime limit	Check <code>pump_allowed</code> , <code>pump_on</code> , and motor PWM telemetry, inspect wiring and driver, and test the pump with the standalone motor test
Excessive pump cooling	Pump dose too large, solution too cold, flow path issue, poor thermal compensation, or incorrect pump timing	Inspect pump timing, flow rate, tubing, solution temperature, and review <code>last_pump_drop_c</code> telemetry
Slow recovery after pump cycle	Heater underpowered, poor mixing, pump disturbance too large, or thermal model mismatch	Check heater PWM, pump recovery time, bath mixing, and pump settings before continuing the test
<code>HEATER_LOCKOUT</code> alarm active	Temperature reached lockout threshold or lockout remains set after a fault	Keep heater disabled, allow bath to cool below reset threshold, inspect cause, and resume only if no critical alarm remains
<code>HARD_KILL</code> alarm active	Temperature reached hard-kill threshold or safety shutdown was triggered	Treat as emergency shutdown, verify heater and pump outputs are off, inspect hardware, and do not restart until root cause is found
<code>MANUAL_KILL</code> alarm active	Operator issued emergency off command or command remains latched	Confirm shutdown state, inspect rig, then release manual kill only when safe to resume
Manual kill command not received	API write key mismatch, ESP32 command polling stopped, backend	Verify <code>API_WRITE_KEY</code> and <code>VITE_API_WRITE_KEY</code> , check ESP32 polling endpoint, confirm backend is reachable, and inspect command queue logs

Issue	Possible Cause	Corrective Action
	command queue issue, or network failure	
Firmware OTA command fails	Wrong firmware file type, unreachable firmware URL, missing reset wiring, bad OTA configuration, or device offline	Verify firmware type, check <code>PUBLIC_BASE_URL</code> , confirm device connectivity, and keep Arduino OTA disabled unless reset/programming path is tested
Email alerts not sent	Email alerts disabled, SMTP settings missing, bad credentials, or cooldown interval active	Check <code>ALERT_EMAIL_ENABLED</code> , SMTP variables, backend logs, and <code>ALERT_EMAIL_COOLDOWN_SECONDS</code>
Cloudflare/public URL not working	Missing tunnel token, tunnel container stopped, DNS route not configured, or service origin mismatch	Set <code>CLOUDFLARE_TUNNEL_TOKEN</code> , check tunnel logs, verify route configuration, and confirm backend/frontend are reachable locally
Tests fail to run	Python dependencies missing, PlatformIO not installed, wrong working directory, or hardware not connected	Install dependencies, run from <code>scc-control-platform</code> , use the documented <code>pytest</code> and <code>pio test</code> commands, and connect required test hardware

Table 30. SCC platform troubleshooting guide.

13.2. Troubleshooting Workflow

When a fault occurs, troubleshooting should follow the sequence below:

1. Review active alarms displayed on the dashboard.
2. Verify that the system is in a safe operating state.
3. Confirm power availability and controller operation.
4. Verify thermistor readings and telemetry updates.

5. Check MQTT communication and backend connectivity.
6. Verify database logging and dashboard operation.
7. Review recent event logs and alarm history.
8. Inspect associated hardware components.
9. Correct the identified fault condition.
10. Perform startup verification before resuming operation.

Following a consistent troubleshooting process helps minimize downtime and reduces the likelihood of overlooking related faults.

13.3. Escalation Criteria

Operation shall be suspended and the system removed from service if any of the following conditions occur:

- Repeated activation of the hard-kill protection system.
- Repeated sensor-fault alarms after sensor replacement.
- Evidence of overheating, smoke, electrical arcing, or damaged wiring.
- Fluid leakage from the SCC cell, tubing, or reservoir.
- Failure of heater lockout or manual kill functions.
- Corrupted database records or persistent telemetry loss.
- Any condition that could compromise operator safety or test validity.

The system shall not be returned to service until the root cause has been identified, corrective action has been completed, and normal operation has been verified.

14. References

- [1] MQTT.org, "MQTT Version 5.0 Specification," MQTT.org, 2019.
- [2] Javan Electronics, "ESP32 DOIT DevKit V1 Technical Reference Manual," Javan Electronics.
- [3] Arduino, "Arduino Uno Rev3 Technical Specifications," Arduino Documentation.
- [4] PostgreSQL Global Development Group, "PostgreSQL Documentation."
- [5] Pallets Projects, "Flask Documentation."
- [6] Docker Inc., "Docker Documentation."
- [7] Eclipse Foundation, "Eclipse Mosquitto Documentation."
- [8] Python Software Foundation, "Python 3.11 Documentation."
- [9] SQLAlchemy Authors, "SQLAlchemy Documentation."
- [10] Socket.IO Authors, "Socket.IO Documentation."
- [11] XGBoost Contributors, "XGBoost Documentation."
- [12] Scikit-learn Developers, "Scikit-learn Documentation."
- [13] Cloudflare Inc., "Cloudflare Tunnel Documentation."
- [14] Intel Corporation, "Intel NUC Technical Product Specifications."
- [15] Microchip Technology Inc., "ATmega328P Datasheet."